

Classificação em Machine Learning: árvores, métricas e KNN

Aprendizado Supervisionado vs. Não Supervisionado

Aprendizado Supervisionado

O aprendizado supervisionado é uma técnica de predição que utiliza dados rotulados para treinar modelos. Ele aprende um mapeamento matemático das entradas para as saídas, permitindo prever valores ou categorias para novos dados.

- Utiliza dados rotulados com exemplos conhecidos
- Aprende relações entre características e resultados
- Aplicado em problemas de predição e classificação

Aprendizado Não Supervisionado

O aprendizado não supervisionado trabalha com dados sem rótulos, descobrindo estruturas e padrões ocultos nos dados. Pode identificar agrupamentos naturais e servir como preparação para tarefas de predição posteriores.

- Utiliza dados não rotulados
- Descobre estruturas e padrões nos dados
- Pode apoiar tarefas de predição subsequentes através de aprendizado de características

Problemas de Predição: Classificação vs. Regressão

Classificação

Prediz rótulos de classe categóricos, sejam eles discretos ou nominais. O processo constrói um modelo baseado no conjunto de treinamento e nos valores dos atributos de classe, utilizando-o posteriormente para classificar novos dados desconhecidos.

Aplicações Típicas



Aprovação de Crédito

Determinar se um cliente deve receber aprovação para empréstimos ou cartões de crédito baseado em seu histórico financeiro.



Detecção de Fraude

Classificar transações como fraudulentas ou legítimas para proteger sistemas financeiros.

Regressão

Modela funções de valores contínuos, permitindo prever valores desconhecidos ou ausentes. Diferente da classificação, a regressão trabalha com variáveis numéricas contínuas ao invés de categorias discretas.



Diagnóstico Médico

Identificar se um tumor é cancerígeno ou benigno através da análise de características clínicas.



Categorização Web

Determinar a categoria apropriada para páginas web, facilitando organização e busca de conteúdo.

Classificação - Um Processo em Duas Etapas

Construção do Modelo

Esta fase envolve a descrição de um conjunto de classes predeterminadas. Cada tupla ou amostra pertence a uma classe predefinida, determinada pelo atributo de rótulo de classe. O conjunto de tuplas usado para construir o modelo é chamado de **conjunto de treinamento**.

- O modelo pode ser representado como regras de classificação, árvores de decisão ou fórmulas matemáticas
- Os dados de treinamento são usados para aprender padrões

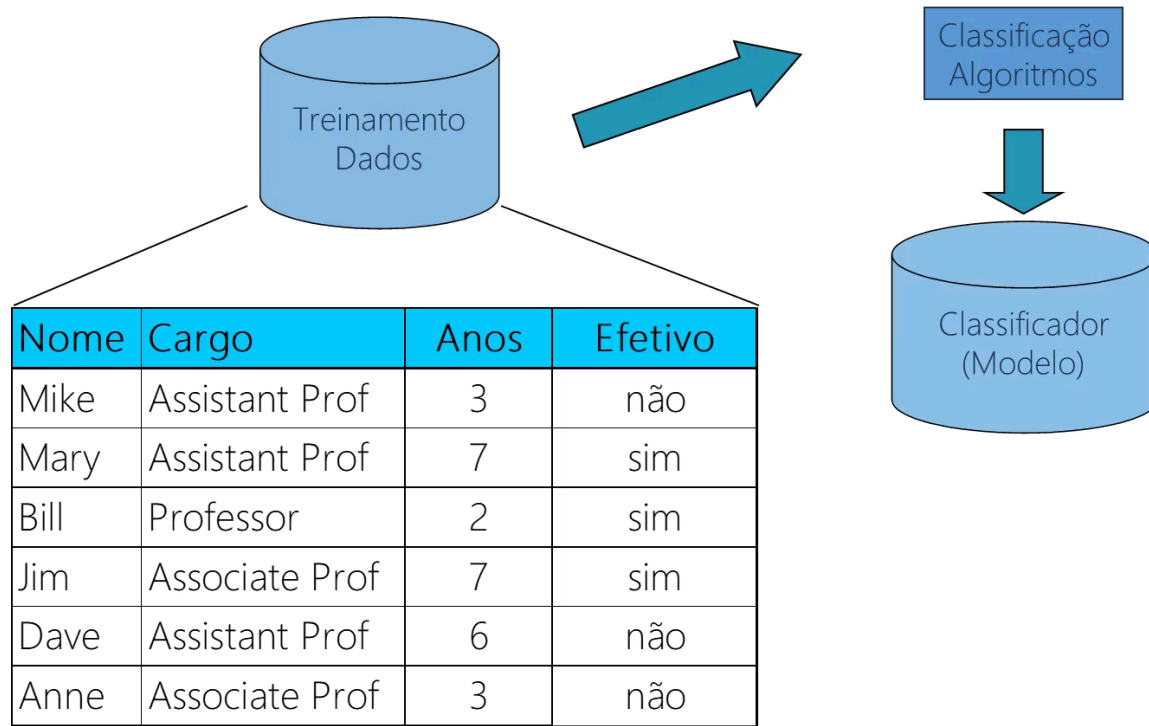
Uso do Modelo

Após a construção, o modelo é aplicado para classificar objetos futuros ou desconhecidos. A acurácia do modelo é estimada comparando o rótulo conhecido de amostras de teste com os resultados classificados.

- **Taxa de acurácia:** percentual de amostras do conjunto de teste corretamente classificadas
- O conjunto de teste deve ser independente do conjunto de treinamento para evitar overfitting
- Se a acurácia for aceitável, o modelo pode ser usado para classificar novos dados

📌 **Nota Importante:** Durante o treinamento, se for necessário selecionar entre diferentes modelos, uma porção do conjunto de treinamento deve ser separada para validação, formando o **conjunto de validação**.

Treinamento do Modelo



O conjunto de treinamento contém exemplos rotulados que o algoritmo utiliza para aprender padrões e relações entre características e classes.

Este exemplo ilustra como um modelo pode aprender regras lógicas a partir dos dados de treinamento, criando condições que determinam a classificação de novos casos.

Algoritmos de Classificação

Processam os dados de treinamento para identificar padrões

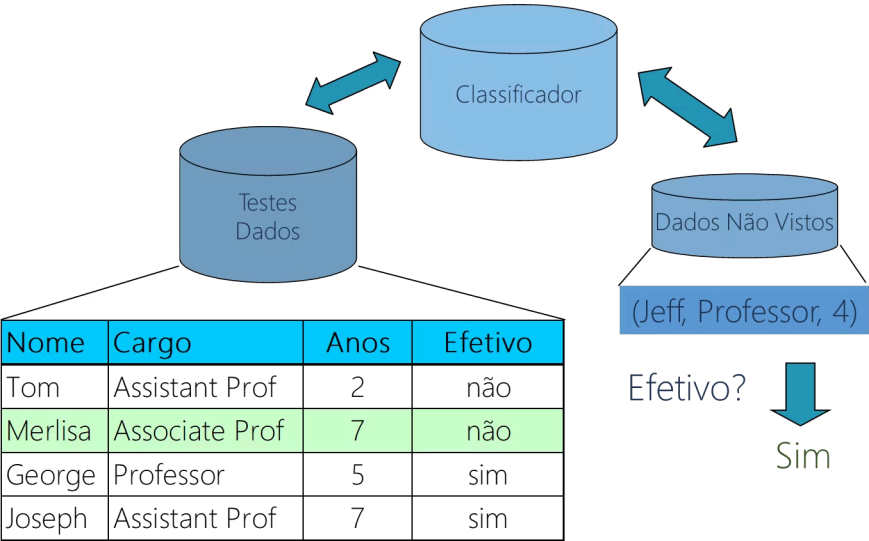
Classificador (Modelo)

Estrutura resultante capaz de fazer previsões

Exemplo de Regra Aprendida:

SE cargo = 'professor'
OU anos > 6
ENTÃO efetivo = 'sim'

Uso do Modelo para Predição



Dados de Entrada

Nome: Merlisa

Cargo: Associate Prof

Anos de Experiência: 7

O modelo recebe um novo caso com características conhecidas, mas sem o rótulo de classe.

Classificação

O modelo aplica as regras aprendidas durante o treinamento para determinar a classe.

Predição: Efetivo?

Com base na regra *SE anos > 6*, o modelo classifica Merlisa como candidato a efetivação.

Este processo demonstra como o modelo generaliza o conhecimento adquirido durante o treinamento para fazer previsões sobre novos casos, aplicando as regras e padrões aprendidos.

Famílias de Métodos de Classificação

Métodos Baseados em Árvore

Decision Trees, Random Forests

Modelos Lineares

Regressão Linear e Logística - métodos que estabelecem relações lineares entre características e classes.

Métodos Probabilísticos

Naïve Bayes

Métodos Baseados em Margem

Support Vector Machines (SVM) - encontram hiperplanos ótimos para separação de classes.

Métodos Baseados em Distância

k-Nearest Neighbors (k-NN)

Métodos Ensemble

Bagging, Boosting, Gradient Boosting - combinam múltiplos modelos para melhorar desempenho.

Cada família de métodos possui características únicas e é mais adequada para diferentes tipos de problemas, estruturas de dados e requisitos de interpretabilidade.

Conjunto de dados de exemplo

Este é um conjunto de dados de exemplo para um problema de classificação. O objetivo é prever se um cliente "Buys Computer" (a classe alvo) com base em características como "Age", "Income", "Student" e "Credit Rating". Este conjunto de dados contém 14 exemplos de treinamento, detalhando as características dos clientes e suas decisões de compra.

Características do Dataset

- 14 instâncias de treinamento
- 4 atributos preditores
- 1 variável alvo binária
- Mix de atributos categóricos

Distribuição das Classes

- **Buys Computer = Yes:** 9 instâncias (64%)
- **Buys Computer = No:** 5 instâncias (36%)

Age	Income	Student	Credit Rating	Buys Computer
≤30	high	no	fair	no
≤30	high	no	excellent	no
31-40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31-40	low	yes	excellent	yes
≤30	medium	no	fair	no
≤30	low	yes	fair	yes
>40	medium	yes	fair	yes
≤30	medium	yes	excellent	yes
31-40	medium	no	excellent	yes
31-40	high	yes	fair	yes
>40	medium	no	excellent	no

Regra Zero – Método Baseline

A **Regra Zero** é o método de classificação mais simples possível, servindo como baseline para comparação. Ela sempre prediz a classe majoritária do conjunto de treinamento, ignorando completamente as características dos dados.

Acurácia Baseline

Percentual da classe majoritária (9 de 14 casos)

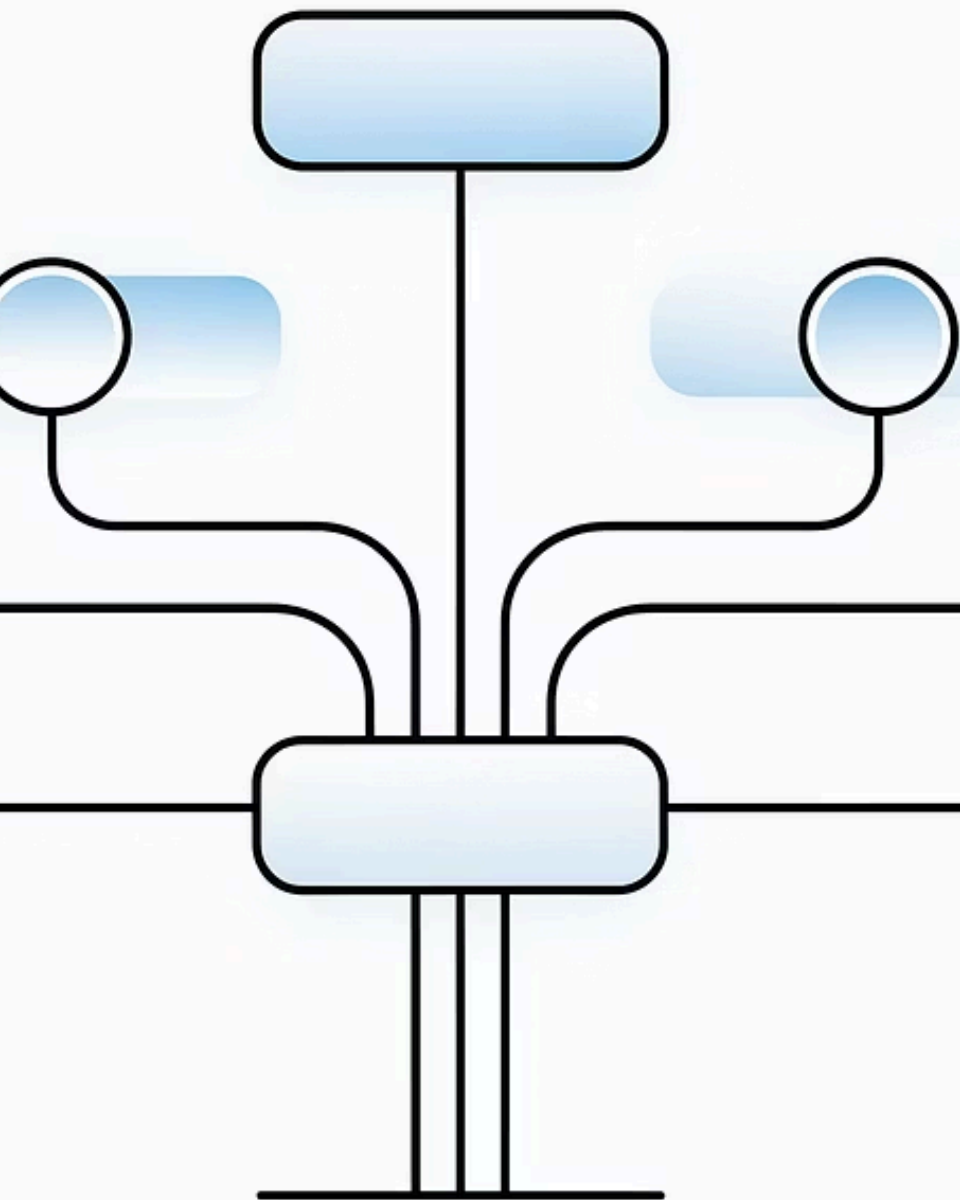
Apesar de sua simplicidade, este método estabelece um ponto de referência importante: qualquer modelo mais sofisticado deve superar esta acurácia mínima para ser considerado útil.

Exemplo: Compra de Computador

Não: 5 casos

Sim: 9 casos

Predição da Regra Zero: Sempre "Sim"

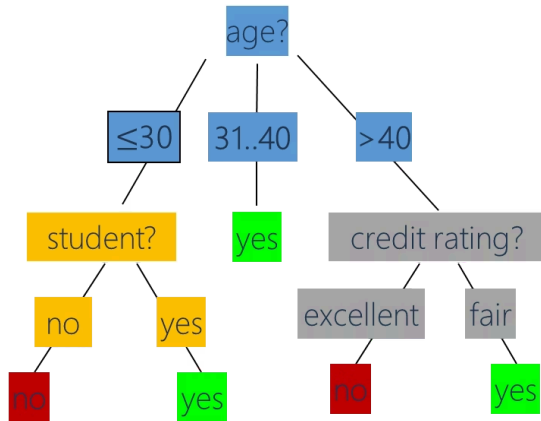


Árvore de Decisão

Explorando um dos métodos mais intuitivos e interpretáveis de classificação: as árvores de decisão.

Algoritmo para Indução de Árvore de Decisão

A árvore é construída de forma **top-down** (de cima para baixo) em um processo recursivo de dividir-e-conquistar. Os atributos devem ser categóricos - se forem contínuos, devem ser discretizados antecipadamente.



Componentes da Árvore

- **Nós Internos:** Representam testes sobre atributos
- **Ramos:** Cada ramificação representa o resultado de um teste
- **Nós Folha:** Representam rótulos de classe - a decisão final

Características Principais

- Estrutura hierárquica clara e interpretável
- Processo de decisão transparente
- Não requer normalização de dados
- Capaz de lidar com dados numéricos e categóricos

Vantagens

- Fácil de entender e visualizar
- Requer pouca preparação de dados
- Pode validar modelos usando testes estatísticos
- Eficiente computacionalmente

ENTRADA: D (conjunto de treinamento), A (lista de atributos), classes C

PRÉ-CONDIÇÃO: atributos categóricos (contínuos devem ser discretizados previamente)

FUNÇÃO GerarÁrvore(D, A):

SE todos os exemplos em D pertencem à **mesma classe c**:

RETORNAR nó-folha rotulado com c

SE A está **vazio**:

RETORNAR nó-folha rotulado com classe majoritária em D

$a^* \leftarrow$ atributo em A com maior Ganho de Informação em relação a D

Criar nó interno N com teste **no** atributo a^*

PARA CADA valor v de a^* :

$D_v \leftarrow$ subconjunto de D onde $a^* = v$

SE D_v está **vazio**:

Adicionar folha com classe majoritária de D como filho de N

SENÃO:

Adicionar GerarÁrvore(D_v , $A \setminus \{a^*\}$) como filho de N

RETORNAR N

SAÍDA: Árvore de decisão T enraizada em N

PÓS-CONDIÇÃO: cada folha de T representa uma classe; cada nó interno representa um teste de atributo

Breve Revisão de Entropia

Conceito de Entropia

A **entropia** é uma medida da impureza ou desordem em um conjunto de dados. Quanto maior a entropia, maior a incerteza sobre qual classe um exemplo pertence.

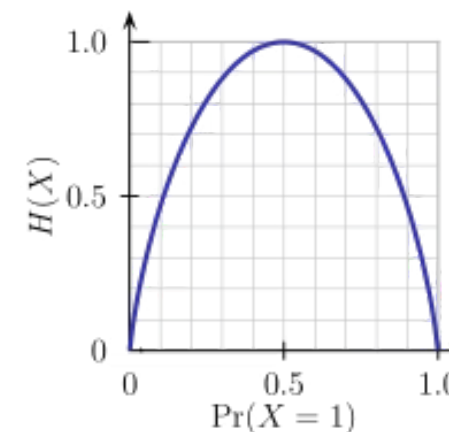
Para uma variável aleatória discreta Y que toma m valores distintos $y_1, \dots, y_m$, tem-se

$$H(Y) = \sum_{i=1}^m p_i \log(p_i)$$

onde $H(Y)$ é a entropia e $p_i = P(Y = y_i)$. A fórmula da entropia pondera a probabilidade de cada classe pelo logaritmo dessa probabilidade, capturando a quantidade de informação necessária para descrever o conjunto de dados.

Quanto maior for a entropia, maior a incerteza.

$$H(Y|X) = \sum_{x \in X} p(x) H(Y|X = x)$$



Exemplo com duas classes (m = 2)

- ❑ **Interpretação Prática:** Em classificação binária, a entropia é máxima (1.0) quando as classes estão perfeitamente balanceadas (50%-50%) e mínima (0.0) quando todos os exemplos pertencem a uma única classe.

Medida de Seleção de Atributos: Ganho de Informação (ID3/C4.5)

O **ganho de informação** é uma medida fundamental usada pelos algoritmos ID3 e C4.5 para determinar qual atributo melhor divide os dados em cada etapa da construção da árvore de decisão.



Cálculo do Ganho

O ganho de informação mede a redução na entropia (desordem) após dividir o conjunto de dados usando um atributo específico. Quanto maior o ganho, melhor o atributo para separar as classes.



Seleção do Melhor Atributo

O algoritmo calcula o ganho de informação para cada atributo disponível e seleciona aquele que proporciona o maior ganho - ou seja, que melhor reduz a incerteza sobre a classe.



Construção Iterativa

Este processo se repete recursivamente para cada subconjunto de dados resultante, construindo a árvore de cima para baixo até que os critérios de parada sejam atingidos.

Fórmula Conceitual: $Ganho_X(Y) = H(Y|X) - H(Y)$

Seleção de Atributos: Ganho de Informação

Conjunto de dados exemplo mostrando diferentes atributos para análise de ganho de informação.

idade	p _i	n _i	H(p _i , n _i)
<=30	2	3	0,971
31...40	4	0	0
>40	3	2	0,971

Classe P: buys computer = "sim" e Classe N: buys computer = "não"

$H(D) = 9/14 \log_2 (9/14) + 5/14 \log_2 (5/14) = -0.940$

Avaliação do atributo de idade

$H(D|age) = 5/14 I(2,3) + 4/14 I(4,0) + 5/14 I(3,2) = -0.694$

$Ganho_{age}(D) = H(D|age) - H(D) = 0.246$

Avaliação de outros atributos

$Ganho_{income}(D) = H(D|income) - H(D) = 0.029$

$Ganho_{student}(D) = H(D|student) - H(D) = 0.151$

$Ganho_{credit, rating}(D) = H(D|credit rating) - H(D) = 0.048$

Este processo sistemático garante que em cada nível da árvore, escolhermos o atributo que mais contribui para classificar corretamente os exemplos, construindo modelos eficientes e interpretáveis.

01

Calcular Entropia Inicial

Medir a impureza do conjunto completo antes de qualquer divisão

02

Simular Divisões

Para cada atributo, calcular a entropia ponderada dos subconjuntos resultantes

03

Determinar Ganho

Subtrair a entropia ponderada da entropia inicial para obter o ganho de informação

04

Escolher Melhor Atributo

Selecionar o atributo com maior ganho de informação para a divisão

Overfitting e Poda de Árvore

O Problema do Overfitting

Uma árvore induzida pode **superajustar** os dados de treinamento, criando ramos excessivos que refletem anomalias devido a ruído ou outliers. Isso resulta em **baixa acurácia para amostras não vistas**, pois o modelo memorizou particularidades dos dados de treinamento ao invés de aprender padrões generalizáveis.

Duas Abordagens para Evitar Overfitting

Pré-Poda (Pre-pruning)

Interrompe a construção da árvore precocemente - não divide um nó se isso resultar na medida de qualidade caindo abaixo de um limiar.

Desafio: Difícil escolher um limiar apropriado que equilibre complexidade e acurácia.

Pós-Poda (Post-pruning)

Remove ramos de uma árvore "completamente crescida", gerando uma sequência de árvores progressivamente podadas.

Vantagem: Usa um conjunto de dados diferente dos dados de treinamento (conjunto de validação) para decidir qual é a "melhor árvore podada".

❏ **Recomendação:** A pós-poda geralmente produz resultados mais confiáveis que a pré-poda, pois toma decisões baseadas na árvore completa e pode avaliar o impacto real de cada ramo no desempenho de generalização.

Outras Medidas de Seleção de Atributos

Além do ganho de informação, existem diversas outras medidas estatísticas e heurísticas para selecionar os melhores atributos durante a construção de árvores de decisão.

Razão de Ganho (Gain Ratio)

Normaliza o ganho de informação pela informação intrínseca do atributo, penalizando atributos com muitos valores distintos. Usado no algoritmo C4.5 para evitar viés em direção a atributos de alta cardinalidade.

Teste Qui-Quadrado

Avalia a independência estatística entre atributos e classes. Útil para determinar quais atributos têm relação significativa com a variável de destino.

A escolha da medida apropriada pode impactar significativamente a estrutura final da árvore e seu desempenho, dependendo das características do problema e dos dados.

Índice Gini

Mede a impureza de um conjunto de dados, calculando a probabilidade de classificar incorretamente um elemento aleatório. Amplamente usado em algoritmos CART (Classification and Regression Trees).

Redução de Variância

Especialmente útil para problemas de regressão, mede quanto a variância é reduzida ao dividir os dados por um atributo específico.

Aprimoramentos à Indução Básica de Árvore de Decisão

Diversos aprimoramentos foram desenvolvidos para tornar as árvores de decisão mais robustas e capazes de lidar com desafios práticos encontrados em dados do mundo real.

Atributos de Valor Contínuo

Define dinamicamente novos atributos discretos que particionam o valor do atributo contínuo em um conjunto discreto de intervalos. Isso permite que árvores de decisão, naturalmente categóricas, trabalhem com dados numéricos contínuos.

Tratamento de Valores Ausentes

Implementa estratégias para lidar com atributos faltantes:

- Atribuir o valor mais comum do atributo no conjunto de dados
- Atribuir probabilidades a cada um dos valores possíveis baseado na distribuição observada

Construção de Atributos

Cria novos atributos baseados em atributos existentes que são esparsamente representados. Esta técnica reduz fragmentação, repetição e replicação na árvore, resultando em modelos mais compactos e interpretáveis.

❏ **Impacto Prático:** Estes aprimoramentos transformam árvores de decisão de algoritmos teóricos em ferramentas práticas capazes de lidar com a complexidade e imperfeições de dados do mundo real.

Classificação em Grandes Bases de Dados

A classificação é um problema clássico extensivamente estudado por estatísticos e pesquisadores de aprendizado de máquina. Em aplicações modernas, a **escalabilidade** tornou-se crucial: classificar conjuntos de dados com milhões de exemplos e centenas de atributos em tempo razoável.

Por que a Indução de Árvore de Decisão é Popular?

Velocidade de Aprendizado

Relativamente mais rápida que outros métodos de classificação, especialmente em grandes volumes de dados

Interpretabilidade

Conversível em regras de classificação simples e fáceis de entender, facilitando validação e aceitação por usuários de negócio

Integração com Bancos de Dados

Pode usar consultas SQL para acessar bancos de dados, facilitando implementação em sistemas empresariais

Acurácia Comparável

Demonstra acurácia de classificação comparável a outros métodos mais complexos na maioria dos problemas práticos

A combinação de eficiência computacional, interpretabilidade e integração com sistemas existentes torna as árvores de decisão uma escolha pragmática para aplicações de classificação em larga escala.

Métricas de Avaliação: Matriz de Confusão

Para avaliar o desempenho real de um classificador, a acurácia tradicional não é suficiente — existem métricas complementares que oferecem uma visão mais completa. É fundamental utilizar um **conjunto de validação separado do treinamento** para garantir uma avaliação honesta e imparcial da capacidade de generalização do modelo.

Estrutura Geral da Matriz

Classe Real \ Predita	C1	¬C1
C1	True Positives (TP)	False Negatives (FN)
¬C1	False Positives (FP)	True Negatives (TN)

Os elementos da diagonal principal (TP e TN) representam as predições corretas. Os demais representam os erros do classificador.

Métodos de Estimativa

- **Holdout** e amostragem aleatória
- **Validação cruzada** (cross-validation)
- **Bootstrap** para estimativas robustas

Exemplo Prático: Compra de Computador

Real \ Predito	yes	no	Total
buy_computer = yes	6954	46	7000
buy_computer = no	412	2588	3000
Total	7366	2634	10000

O modelo acertou 6954 casos positivos e 2588 negativos — totalizando 9542 predições corretas em 10000 instâncias.

Comparação de Classificadores

- Intervalos de confiança estatísticos
- Análise de custo-benefício
- Curvas ROC para comparação visual

Acurácia, Taxa de Erro, Sensibilidade e Especificidade

Acurácia (Accuracy)

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{All}$$

Proporção de predições corretas em relação ao total de casos. Mede o desempenho geral do classificador, mas pode ser enganosa em datasets desbalanceados.

Taxa de Erro (Error Rate)

$$\text{Error rate} = (\text{FP} + \text{FN}) / \text{All}$$

Proporção de predições incorretas. É o complemento da acurácia e indica a frequência com que o modelo comete erros.

Sensibilidade (Sensitivity)

$$\text{Sensitivity} = \text{TP} / \text{P} = \text{TP} / (\text{TP} + \text{FN})$$

Taxa de verdadeiros positivos. Mede a capacidade do modelo de identificar corretamente os casos positivos. Também conhecida como *recall* ou *true positive rate*.

Especificidade (Specificity)

$$\text{Specificity} = \text{TN} / \text{N} = \text{TN} / (\text{FP} + \text{TN})$$

Taxa de verdadeiros negativos. Mede a capacidade do modelo de identificar corretamente os casos negativos. Complementa a sensibilidade na avaliação completa do desempenho.

❏ **Referência da Matriz:** A (Actual) = Classe Real, P (Predicted) = Classe Predita. P = Total de Positivos Reais, N = Total de Negativos Reais, P' = Total Predito como Positivo, N' = Total Predito como Negativo.

Precisão, Revocação e Medidas-F

Estas métricas são especialmente importantes quando trabalhamos com datasets desbalanceados ou quando os custos de falsos positivos e falsos negativos são diferentes.

Precisão (Precision)

$$Precision = \frac{TP}{(TP+FP)}$$

Proporção de instâncias classificadas como positivas que são realmente positivas. Responde: "Das instâncias que o modelo previu como positivas, quantas realmente são?"

Medida-F (F-measure)

$$F_1 = 2 \times \frac{Precision \times Recall}{(Precision + Recall)}$$

Média harmônica entre precisão e revocação. Fornece uma medida única que equilibra ambas as métricas, sendo especialmente útil quando precisamos de um único número para comparar modelos.

Revocação (Recall)

$$Recall = \frac{TP}{(TP+FN)}$$

Proporção de instâncias positivas reais que foram identificadas corretamente. Responde: "De todas as instâncias positivas reais, quantas o modelo conseguiu identificar?"

F_β - Medida Ponderada

$$F_\beta = (1 + \beta^2) \times \frac{Precision \times Recall}{\beta^2 \times (Precision + Recall)}$$

Versão generalizada que permite dar mais peso à precisão ($\beta < 1$) ou à revocação ($\beta > 1$). O parâmetro β ajusta a importância relativa de cada métrica conforme o contexto da aplicação.

Visualizando Sensibilidade e Especificidade

Matriz de Referência

A \ P	C	¬C
C	TP	FN
¬C	FP	TN
	P'	N'

Sensibilidade Ideal

Quando TP = P, ou seja, todos os casos positivos são identificados corretamente (FN = 0)

Especificidade Ideal

Quando TN = N, ou seja, todos os casos negativos são identificados corretamente (FP = 0)

Sensibilidade Alta

Minimiza falsos negativos. Essencial em diagnósticos médicos onde não detectar uma doença pode ser fatal.

Trade-off

Aumentar sensibilidade geralmente reduz especificidade e vice-versa. O equilíbrio depende do contexto.

Especificidade Alta

Minimiza falsos positivos. Importante quando tratamentos desnecessários têm alto custo ou risco.

Caso Prático: Detecção de Câncer

Vamos analisar um exemplo concreto de classificação de câncer em uma população de 10.000 pacientes, demonstrando como interpretar as métricas na prática.

Matriz de Confusão

Real \ Predito	yes	no	Total
cancer = yes	90	210	300
cancer = no	140	9560	9700
Total	230	9770	10000

O modelo identificou corretamente 9650 casos (90 + 9560), mas falhou em 350 casos (210 + 140).

96.40%

Acurácia

Taxa geral de acertos

30.00%

Sensibilidade

Recall = 90/300

98.56%

Especificidade

TN rate = 9560/9700

39.13%

Precisão

Precision = 90/230

Interpretação Crítica dos Resultados

❏ **Problema Identificado:** Embora a acurácia seja alta (96.40%), a **sensibilidade é apenas 30%**. Isso significa que o modelo falha em detectar **70% dos casos de câncer** (210 de 300 pacientes doentes).

Implicações Práticas: Em diagnósticos médicos, uma sensibilidade tão baixa é inaceitável, pois resulta em muitos falsos negativos. Pacientes com câncer seriam enviados para casa sem tratamento, com consequências potencialmente fatais.

Métodos de Validação - Holdout e Validação Cruzada

Diferentes estratégias de divisão dos dados permitem estimar a capacidade de generalização do modelo de forma mais confiável e robusta.

Holdout Method

Divisão simples dos dados em conjuntos de treino e teste (ex: 70%/30% ou 80%/20%). Rápido, mas a estimativa pode variar dependendo da divisão aleatória escolhida.

Random Subsampling

Repetição do holdout múltiplas vezes com divisões aleatórias diferentes. A acurácia final é a média das iterações, fornecendo uma estimativa mais estável.

K-Fold Cross-Validation

Divisão dos dados em K partes (folds) de tamanho igual. O modelo é treinado K vezes, cada vez usando K-1 folds para treino e 1 fold para teste. Fornece estimativa robusta com uso eficiente dos dados.

Leave-One-Out (LOO)

Caso especial de K-fold onde $K = N$ (número total de instâncias). Cada instância é usada uma vez como teste. Máximo aproveitamento dos dados, mas computacionalmente caro para datasets grandes.

Vantagens da Validação Cruzada

- Uso mais eficiente dos dados disponíveis
- Estimativa mais confiável do erro de generalização
- Reduz variância causada por divisão aleatória
- Permite avaliar estabilidade do modelo

Considerações Práticas

- **K=5 ou K=10:** Valores comuns que equilibram viés e variância
- **Stratified K-Fold:** Mantém proporção de classes em cada fold
- **Custo computacional:** Aumenta linearmente com K

Método Bootstrap - Avaliação com Bootstrap

Conceito Fundamental

Bootstrap é um método de reamostragem que cria múltiplos datasets de treino através de amostragem *com reposição* do dataset original. Cada instância pode aparecer zero, uma ou múltiplas vezes em cada amostra bootstrap.

Como Funciona o Bootstrap

1. Dado um dataset D com n instâncias
2. Criar uma amostra bootstrap D_i selecionando n instâncias aleatoriamente **com reposição** de D
3. Treinar o modelo usando D_i
4. Testar nas instâncias de D que não aparecem em D_i (out-of-bag samples)
5. Repetir o processo múltiplas vezes (tipicamente 50-200 iterações)
6. Calcular a média das acurácias obtidas

Probabilidade Out-of-Bag

Matematicamente, a probabilidade de uma instância **não** ser selecionada em uma amostra bootstrap é aproximadamente:

$$(1 - 1/n)^n \approx 0.368 \text{ ou } 36.8\%$$

Portanto, cerca de 63.2% das instâncias são usadas para treino e 36.8% para teste em cada iteração.

Vantagens do Bootstrap

- Funciona bem com datasets pequenos
- Fornece estimativas de intervalo de confiança
- Permite avaliar variabilidade do modelo
- Não requer estratificação explícita

Limitações

- Viés de estimativa (ligeiramente pessimista)
- Maior custo computacional que holdout
- Amostras de treino e teste não são independentes
- Pode ser problemático com dados temporais

📌 **Bootstrap .632:** Uma variação popular que corrige o viés combinando a taxa de erro bootstrap com a taxa de erro de ressubstituição: $\text{Error} = 0.632 \times \text{err_bootstrap} + 0.368 \times \text{err_training}$

Seleção de Modelos com Curvas ROC

As curvas ROC (Receiver Operating Characteristics) fornecem uma ferramenta visual poderosa para comparar classificadores e escolher o limiar de decisão ideal, originadas da teoria de detecção de sinais.

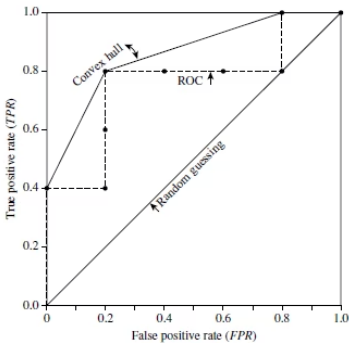
Construção da Curva ROC

1. Ordenar tuplas de teste em ordem decrescente pela probabilidade de pertencer à classe positiva
2. Para cada possível limiar de decisão, calcular:
 - **TPR** (True Positive Rate) = Sensibilidade
 - **FPR** (False Positive Rate) = 1 - Especificidade
3. Plotar TPR (eixo vertical) versus FPR (eixo horizontal)
4. Calcular a área sob a curva (AUC)

Tuple #	Class	Prob.	TP	FP	TN	FN	TPR	FPR
1	P	0.90	1	0	5	4	0.2	0
2	P	0.80	2	0	5	3	0.4	0
3	N	0.70	2	1	4	3	0.4	0.2
4	P	0.60	3	1	4	2	0.6	0.2
5	P	0.55	4	1	4	1	0.8	0.2
6	N	0.54	4	2	3	1	0.8	0.4
7	N	0.53	4	3	2	1	0.8	0.6
8	N	0.51	4	4	1	1	0.8	0.8
9	P	0.50	5	4	0	1	1.0	0.8
10	N	0.40	5	5	0	0	1.0	1.0

Interpretação da Curva

A curva ROC mostra o **trade-off** entre a taxa de verdadeiros positivos e a taxa de falsos positivos à medida que variamos o limiar de decisão.



- AUC = 1.0 indica classificador perfeito
- AUC = 0.5 equivale a classificação aleatória (linha diagonal)

Área Sob a Curva (AUC)

A AUC é uma métrica única que resume o desempenho do classificador em todos os possíveis limiares. Quanto mais próxima de 1.0, melhor o modelo. Uma AUC próxima de 0.5 indica que o modelo não é melhor que o acaso.

Comparação Visual de Modelos

A curva que está mais próxima do canto superior esquerdo (TPR=1, FPR=0) representa o melhor classificador. Podemos comparar múltiplos modelos plotando suas curvas ROC no mesmo gráfico.

Aplicações Práticas

Curvas ROC são especialmente úteis quando: (1) o dataset é desbalanceado, (2) os custos de diferentes tipos de erro variam, ou (3) precisamos escolher um limiar de decisão customizado para a aplicação.

Métricas para Regressão - Medidas de Erro para Preditores

Enquanto classificadores lidam com variáveis categóricas, preditores lidam com variáveis contínuas. As métricas de avaliação precisam capturar a magnitude e direção dos erros de predição.

MAE - Mean Absolute Error

$$MAE = \frac{\sum_{i=1}^d |y_i - \hat{y}_i|}{d}$$

Média dos valores absolutos dos erros. Fornece uma medida direta e interpretável do erro médio, na mesma unidade da variável alvo. Menos sensível a outliers que MSE.

MSE - Mean Squared Error

$$MSE = \frac{\sum_{i=1}^d (y_i - \hat{y}_i)^2}{d}$$

Média dos quadrados dos erros. Penaliza erros maiores mais fortemente devido à elevação ao quadrado. É a métrica mais comum em otimização, mas em unidades quadradas.

RMSE - Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

Raiz quadrada do MSE, retornando a métrica à unidade original. Combina as vantagens do MSE (penalização de erros grandes) com interpretabilidade do MAE.

MAPE - Mean Absolute Percentage Error

$$MAPE = \frac{100}{n} \sum_{i=1}^d \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Erro percentual médio. Útil para comparar erros entre diferentes escalas de dados. Cuidado: não definido quando $y_i = 0$ e sensível a valores pequenos de y_i .

Escolhendo a Métrica Adequada

- **MAE:** Quando todos os erros são igualmente importantes
- **MSE/RMSE:** Quando erros grandes são particularmente prejudiciais
- **MAPE:** Para comparações entre diferentes escalas
- **R² (coeficiente de determinação):** Para medir variância explicada

❏ **Importante:** Ao reportar resultados, sempre especifique qual métrica de erro está sendo utilizada, pois diferentes métricas podem levar a conclusões diferentes sobre o desempenho do modelo.

Fatores que Afetam a Seleção de Modelos

A escolha do melhor modelo vai além da simples acurácia. Diversos fatores práticos e contextuais devem ser considerados para selecionar o classificador mais adequado para cada aplicação específica.

Acurácia

Capacidade de prever corretamente rótulos de classe. Deve ser avaliada com métricas apropriadas ao contexto (acurácia, precisão, recall, F-measure, AUC-ROC).

Velocidade

Tempo para construir o modelo (treino) e tempo para usar o modelo (predição). Crítico em aplicações de tempo real ou com grandes volumes de dados.

Robustez

Capacidade de lidar com ruído e valores ausentes nos dados sem degradação significativa de desempenho. Importante para dados do mundo real.

Interpretabilidade

Compreensão e insights fornecidos pelo modelo. Fundamental em domínios regulados (medicina, finanças) onde decisões precisam ser explicáveis.

Escalabilidade

Eficiência ao trabalhar com grandes volumes de dados armazenados em disco. Essencial para aplicações com datasets massivos ou em crescimento contínuo.

Outras Medidas

Qualidade das regras geradas, tamanho da árvore de decisão, compacidade das regras de classificação. Afetam manutenibilidade e implantação.

Trade-offs Comuns

- **Acurácia vs. Interpretabilidade:** Modelos mais complexos geralmente são mais precisos mas menos interpretáveis
- **Velocidade vs. Acurácia:** Modelos simples são rápidos mas podem sacrificar performance
- **Robustez vs. Especificidade:** Modelos muito específicos podem ser sensíveis a ruído

Recomendação Prática

Não existe um "melhor" modelo universal. A escolha deve ser guiada pelos requisitos específicos da aplicação, considerando o contexto de negócio, restrições técnicas, e necessidades dos stakeholders. Sempre valide múltiplos modelos antes de decidir.

Classificação Bayesiana

Um classificador estatístico que realiza previsões probabilísticas, ou seja, prediz probabilidades de pertencimento a classes. Este método poderoso combina conhecimento prévio com dados observados para tomar decisões informadas.

Base Teórica

Fundamentado no Teorema de Bayes, um dos pilares da inferência estatística moderna.

Performance

O classificador Bayesiano ingênuo (naïve) apresenta desempenho comparável a árvores de decisão e redes neurais selecionadas.

Aprendizado Incremental

Cada exemplo de treinamento pode incrementalmente aumentar ou diminuir a probabilidade de que uma hipótese esteja correta.

Mesmo quando os métodos Bayesianos são computacionalmente intratáveis, eles podem fornecer um padrão de tomada de decisão ideal contra o qual outros métodos podem ser medidos.

Classificação Como Máximo A Posteriori

O objetivo da classificação Bayesiana é derivar o **Maximum A Posteriori (MAP)** - a hipótese mais provável dado os dados observados.

Teorema de Bayes: Fundamentos

$P(H)$ – Prior

Probabilidade inicial da hipótese antes de observar os dados.

$P(X)$ – Evidência

Probabilidade marginal dos dados observados.

$P(H|X)$ – Posterior

Probabilidade da hipótese após observar os dados.

$P(X|H)$ – Verossimilhança

Probabilidade dos dados dado que a hipótese é verdadeira.

$$P(H|X) = \frac{P(X|H) \cdot P(H)}{P(X)}$$

Passos da Predição

Definir Prior

Estimar $P(c_i)$ para cada classe a partir dos dados de treinamento.

Calcular Verossimilhança

Determinar $P(X|c_i)$ para cada classe c_i .

Aplicar Bayes

Calcular a probabilidade posterior $P(c_i|X)$.

Classificar

Atribuir a instância à classe com maior probabilidade posterior.

Pseudocódigo: ClassificarMAP

ENTRADA: $X = (x_1, x_2, \dots, x_n)$ (instância a classificar)

$C = \{c_1, c_2, \dots, c_m\}$ (classes)

PRÉ-CONDIÇÃO: probabilidades $P(c_i)$ e $P(X|c_i)$ estimadas a partir do conjunto de treinamento

FUNÇÃO **ClassificarMAP**(X, C):

PARA CADA classe c_i em C :

Regra de Bayes: $P(c_i|X) = P(X|c_i) \times P(c_i) / P(X)$

$P(X)$ é constante → pode ser ignorado na comparação

score(c_i) ← $P(X|c_i) \times P(c_i)$

$c_MAP \leftarrow \operatorname{argmax}_{c_i} \text{score}(c_i)$

RETORNAR c_MAP # classe com maior probabilidade a posteriori

SAÍDA: c_MAP (hipótese MAP — classe mais provável dado X)

PÓS-CONDIÇÃO: se $P(X|c_i)$ for difícil de calcular diretamente, aplicar suposição de independência (Naïve Bayes)

Classificador Naïve Bayes

O classificador Naïve Bayes simplifica o cálculo das probabilidades fazendo uma suposição *ingênua* mas poderosa: todas as características são condicionalmente independentes dada a classe.

Assume -se independência das variáveis: $P(X|c_i) = P(x_1|c_i) \times P(x_2|c_i) \times \dots \times P(x_n|c_i)$, onde $x_1, x_2, \dots, x_n$ são os valores dos atributos.

ENTRADA: D (conjunto de treinamento), $X = (x_1, x_2, \dots, x_n)$ (nova instância a classificar)

PRÉ-CONDIÇÃO: atributos condicionalmente independentes dada a classe (suposição naïve)

FASE 1 — Treinamento (calcular probabilidades a partir de D):

PARA CADA classe c_i em C :

$P(c_i) \leftarrow |D \text{ com classe } c_i| / |D|$ # probabilidade prévia

PARA CADA atributo x_k :

$P(x_k | c_i) \leftarrow \text{freq}(x_k, c_i) / |D \text{ com classe } c_i|$ # prob. condicional

FASE 2 — Classificação de X :

PARA CADA classe c_i em C :

$\text{score}(c_i) \leftarrow P(c_i) \times P(x_1 | c_i) \times P(x_2 | c_i) \times \dots \times P(x_n | c_i)$

$\text{classe_predita} \leftarrow \text{argmax_ci } \text{score}(c_i)$

SAÍDA: classe_predita (classe com maior probabilidade a posteriori)

PÓS-CONDIÇÃO: se $P(x_k | c_i) = 0$ para algum atributo, aplicar suavização de Laplace

Exemplo de Classificação Naïve Bayes

Vamos classificar uma nova instância: **X = (age = ≤30, income = medium, student = yes, credit_rating = fair)**

1

Passo 1: Calcular $P(\text{buys_computer})$

$$P(\text{yes}) = 9/14 = 0.643$$

$$P(\text{no}) = 5/14 = 0.357$$

2

Passo 2: Calcular $P(X|\text{yes})$

$$\begin{aligned} &P(\text{age} \leq 30 | \text{yes}) \times P(\text{income} = \text{medium} | \text{yes}) \times P(\text{student} = \text{yes} | \text{yes}) \times P(\text{credit_rating} = \text{fair} | \text{yes}) \\ &= (2/9) \times (4/9) \times (6/9) \times (6/9) = 0.044 \end{aligned}$$

3

Passo 3: Calcular $P(X|\text{no})$

$$\begin{aligned} &P(\text{age} \leq 30 | \text{no}) \times P(\text{income} = \text{medium} | \text{no}) \times P(\text{student} = \text{yes} | \text{no}) \times P(\text{credit_rating} = \text{fair} | \text{no}) \\ &= (3/5) \times (2/5) \times (1/5) \times (2/5) = 0.019 \end{aligned}$$

4

Passo 4: Comparar Probabilidades

$$P(X|\text{yes}) \times P(\text{yes}) = 0.044 \times 0.643 = 0.028$$

$$P(X|\text{no}) \times P(\text{no}) = 0.019 \times 0.357 = 0.007$$



Resultado: Como $P(X|\text{yes}) \times P(\text{yes}) > P(X|\text{no}) \times P(\text{no})$, classificamos X como $\text{buys_computer} = \text{YES}$

Comentários sobre o Classificador Naïve Bayes

Vantagens

Simplicidade de Implementação

Fácil de implementar e entender, ideal para prototipagem rápida e baseline de projetos.

Eficiência Computacional

Requer apenas uma passagem pelos dados de treinamento, com tempo linear de classificação.

Bons Resultados

Na maioria dos casos, obtém resultados competitivos mesmo com a suposição simplificada.

Robusto a Dados Irrelevantes

Funciona bem mesmo com muitos atributos irrelevantes no dataset.

Desvantagens

Suposição Forte

A suposição de independência condicional entre classes raramente é verdadeira na prática.

Perda de Acurácia

Quando existem dependências fortes entre atributos, a performance pode degradar significativamente.

Problema de Probabilidade Zero

Se um atributo não aparece no treinamento para uma classe, $P(X|C) = 0$ (resolvido com suavização de Laplace).

Estimativas de Probabilidade

As probabilidades estimadas podem não ser bem calibradas, embora a classificação ainda seja precisa.

Apesar das limitações teóricas, o Naïve Bayes continua sendo uma escolha popular devido à sua simplicidade, eficiência e surpreendente eficácia em muitas aplicações do mundo real.

Regressão Logística

A regressão logística é uma técnica fundamental de aprendizado de máquina para problemas de classificação. Ao contrário da regressão linear, que prevê valores contínuos, a regressão logística é projetada especificamente para prever categorias ou classes discretas.

Este método estatístico poderoso transforma uma combinação linear de variáveis de entrada em probabilidades, permitindo classificar observações em diferentes grupos. É amplamente utilizado em diversas áreas, desde diagnóstico médico até detecção de fraudes e previsão de comportamento de clientes.

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
numeric	numeric	numeric	numeric	factor
Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa

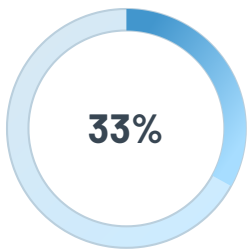
Simplificando o Problema

Foco na Predição de Uma Classe

Para facilitar a compreensão e implementação inicial, começamos com um problema de classificação binária. Esta abordagem reduz a complexidade multiclasse para uma decisão simples: a observação pertence ou não à classe de interesse.

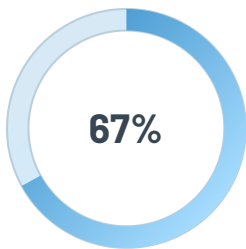
No exemplo clássico do dataset Iris, podemos focar na identificação da espécie **versicolor** versus todas as outras espécies (não-versicolor).

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species	versicolor
31	4.8	3.1	1.6	0.2	other	0
27	5.0	3.4	1.6	0.4	other	0
102	5.8	2.7	5.1	1.9	other	0
57	6.3	3.3	4.7	1.6	versicolor	1
113	6.8	3.0	5.5	2.1	other	0
73	6.3	2.5	4.9	1.5	versicolor	1



Versicolor

Classe positiva de interesse



Não-Versicolor

Todas as outras classes

Esta transformação binária nos permite utilizar a regressão logística em sua forma mais direta, estabelecendo uma base sólida antes de expandir para problemas multiclasse mais complexos.

Construindo o Modelo: Regressão Logística

Creation of logistic regression model using all independent variables.

```
pred <- glm(versicolor ~ .-Species, data=train, family = binomial)
```

Measuring the level of adjustment using training data.

```
res <- predict(pred, train, type="response")
res <- as.integer(res >= t)
table(res, train$versicolor)
```

```
res  0  1
    60  9
    20 31
```

ENTRADA: Dataset com variáveis preditoras $X = \{x_1, x_2, \dots, x_n\}$ e rótulos binários $y \in \{0, 1\}$ (versicolor ou não)

PRÉ-CONDIÇÃO: Variável alvo *Species* excluída de X ; dados normalizados; parâmetros iniciais $\beta = 0$

PASSOS:

1. Selecionar todas as variáveis preditoras exceto *Species*
2. Calcular a combinação linear: $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$
3. Aplicar a função sigmoide: $p = 1 / (1 + e^{-z})$
4. Calcular a log-verossimilhança (MLE) ou o custo (gradiente descendente)
5. Atualizar coeficientes β para minimizar o erro de previsão
6. Repetir passos 2–5 até convergência
7. Avaliar ajuste com métricas (AIC, deviance, acurácia)

SAÍDA: Coeficientes ajustados β e probabilidades preditas $\hat{p}$ para cada observação

PÓS-CONDIÇÃO: Modelo capaz de mapear características de entrada em probabilidade de pertencer à classe *versicolor*

Medindo o Desempenho do Modelo

Avaliação das Predições

Após treinar o modelo com todas as variáveis, é crucial avaliar sua capacidade de generalização. Aplicamos o modelo a dados de teste para obter predições e comparar com os valores reais conhecidos.

As métricas de desempenho revelam não apenas a acurácia geral, mas também erros específicos como falsos positivos e falsos negativos, essenciais para entender as limitações do modelo.

```
res <- predict(pred, test, type="response")
res <- res >= t
table(res, test$versicolor)
```

res	0	1
FALSE	15	6
TRUE	5	4

Acurácia

Proporção de predições corretas sobre o total

Precisão e Recall

Equilíbrio entre falsos positivos e negativos

Matriz de Confusão

Visualização detalhada dos acertos e erros

Construindo um Modelo Mais Simples

Frequentemente, modelos mais simples podem oferecer desempenho comparável com melhor interpretabilidade. A análise exploratória de dados e o pré-processamento fornecem insights valiosos sobre quais variáveis são mais relevantes.

Seleção Baseada em Evidências

- **Petal.Length** e **Petal.Width** mostraram maior significância estatística na análise exploratória inicial
- Durante o pré-processamento, essas variáveis resultaram em menor entropia durante a discretização
- Indicadores de maior poder discriminativo para separar as classes

📌 **Princípio da Parcimônia:** Modelos mais simples são preferíveis quando oferecem desempenho similar, pois são mais fáceis de interpretar, menos propensos a overfitting e mais eficientes computacionalmente.

Medindo o Desempenho do Modelo Simplificado

Modelo Completo

Utiliza todas as 4 variáveis disponíveis

Análise Comparativa

Avaliação lado a lado do desempenho

Modelo Simplificado

Apenas 2 variáveis mais relevantes

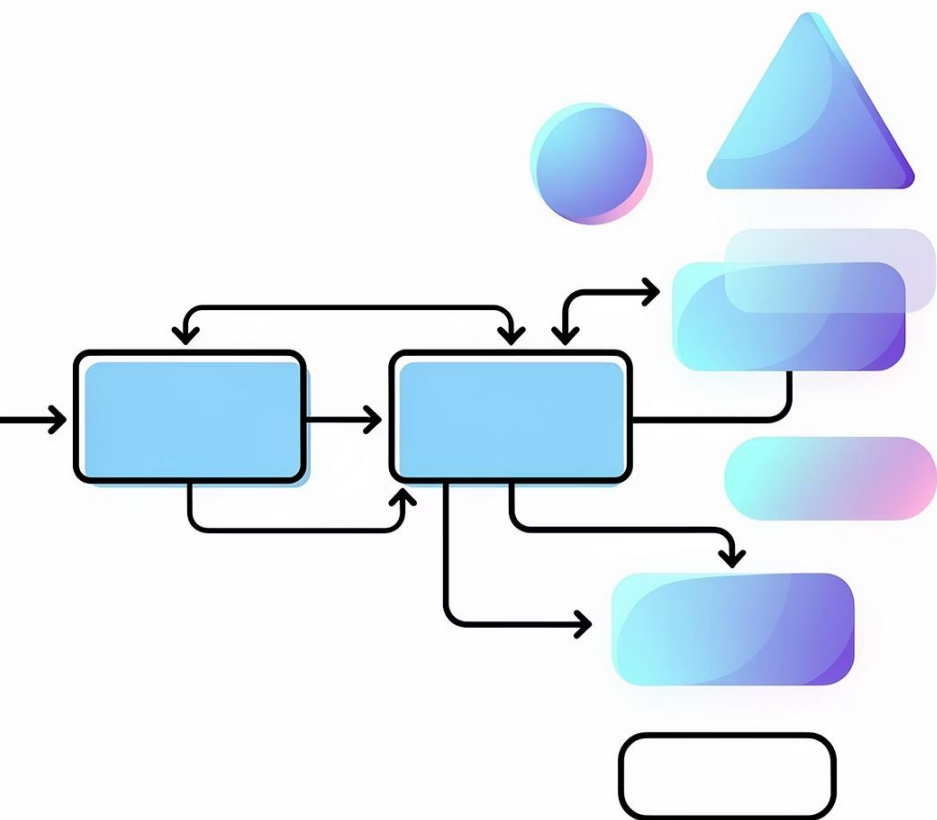
Resultados da Predição

O modelo simplificado, usando apenas Petal.Length e Petal.Width, demonstra que a redução de complexidade não necessariamente compromete o desempenho. Em muitos casos, pode até melhorar a generalização ao eliminar ruído e correlações espúrias.

Compare as métricas de desempenho entre os dois modelos para determinar se a simplicidade adicional justifica qualquer pequena perda de acurácia. Geralmente, uma diferença mínima favorece o modelo mais simples.

```
res <- predict(pred, test, type="response")
res <- as.integer(res >= t)
table(res, test$versicolor)
```

```
res  0  1
  0 16  2
  1  4  8
```



Classificação por Aprendizagem Preguiçosa

Mudando de paradigma: explorando métodos que adiam o processamento até o momento da predição, contrastando com as abordagens ansiosas que aprendemos anteriormente.

Aprendizagem Preguiçosa vs. Ansiosa

A diferença fundamental entre esses dois paradigmas está em *quando* o modelo é construído e *como* as previsões são realizadas.

Aprendizagem Ansiosa (Eager Learning)

Métodos como Naïve Bayes, árvores de decisão e redes neurais constroem um modelo de classificação **antes** de receber novos dados para classificar.

- Mais tempo no treinamento
- Menos tempo na predição
- Compromete-se com uma única hipótese global

Aprendizagem Preguiçosa (Lazy Learning)

Métodos baseados em instâncias simplesmente armazenam os dados de treinamento e **esperam** até receber uma tupla de teste para classificar.

- Menos tempo no treinamento
- Mais tempo na predição
- Usa múltiplas funções lineares locais

Vantagem da Acurácia

Métodos preguiçosos efetivamente usam um espaço de hipóteses mais rico, pois utilizam muitas funções lineares locais para formar uma aproximação global implícita à função alvo.

- ❑ **Trade-off Chave:** A aprendizagem preguiçosa sacrifica velocidade de predição por maior flexibilidade e potencial de acurácia, especialmente em espaços de decisão complexos.

Métodos Baseados em Instâncias

A aprendizagem baseada em instâncias é uma família de algoritmos que armazenam exemplos de treinamento e adiam o processamento até que uma nova instância precise ser classificada.

Princípio Fundamental

Em vez de abstrair os dados em um modelo, os métodos baseados em instâncias mantêm os exemplos em memória e os usam diretamente para fazer previsões. A classificação é baseada na **similaridade** entre instâncias.

k-Nearest Neighbors (k-NN)

As instâncias são representadas como pontos em um espaço Euclidiano. A classificação é baseada na maioria de votos dos k vizinhos mais próximos.

Regressão Ponderada Localmente

Constrói aproximações locais usando regressão ponderada pelos vizinhos. Pontos mais próximos têm maior influência na previsão.

Raciocínio Baseado em Casos

Usa representações simbólicas e inferência baseada em conhecimento. Recupera casos similares e adapta suas soluções ao novo problema.

Cada abordagem oferece diferentes trade-offs entre complexidade computacional, interpretabilidade e acurácia. O k-NN é o método mais popular devido à sua simplicidade e eficácia.

O Algoritmo k-Nearest Neighbors

Conceito Central

O k-NN classifica uma nova instância com base nas classes de seus k vizinhos mais próximos no espaço de características.

ENTRADA: D (conjunto de treinamento rotulado), x_q (nova instância),

k (número de vizinhos), f (função de distância)

PRÉ-CONDIÇÃO: $k \geq 1$, atributos numéricos ou com distância definida

FUNÇÃO $kNN(D, x_q, k)$:

PARA CADA instância x_i em D:

$dist[i] \leftarrow f(x_q, x_i)$ # ex.: distância Euclidiana

$vizinhos \leftarrow k$ instâncias de D com menor $dist[]$

PARA CADA classe c em C:

$votos[c] \leftarrow |\{x_i \in vizinhos : classe(x_i) = c\}|$

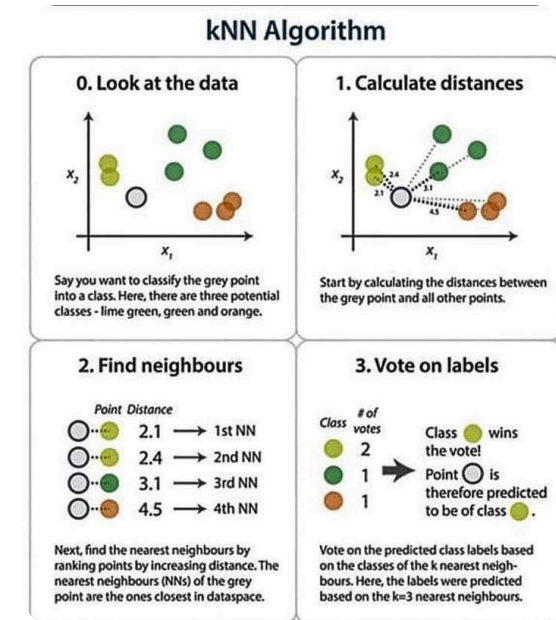
$classe_predita \leftarrow \text{argmax}_c votos[c]$ # votação majoritária

RETORNAR $classe_predita$

SAÍDA: $classe_predita$ atribuída a x_q

PÓS-CONDIÇÃO: empates resolvidos por distância mínima ou k ímpar

NOTA: k pequeno $\rightarrow$ sensível a ruído; k grande $\rightarrow$ fronteiras mais suaves



Visualização do k-NN

Esta visualização mostra como o algoritmo k-NN classifica um novo ponto x_q com base em seus vizinhos mais próximos. Os símbolos representam diferentes classes: círculos (+), quadrados (-) e triângulos (.).

Ponto de Consulta

x_q é a nova instância que queremos classificar

Cálculo de Distância

Distância Euclidiana calculada para todos os pontos de treinamento

Seleção de Vizinhos

Os k pontos mais próximos são identificados

Votação por Maioria

A classe mais frequente entre os k vizinhos determina a classificação

Fronteiras de Decisão

As fronteiras entre classes no k-NN são **não lineares** e **localmente adaptativas**, permitindo capturar padrões complexos nos dados.

Influência do Valor k

Valores menores de k criam fronteiras mais complexas e irregulares. Valores maiores produzem fronteiras mais suaves e generalizadas.

Discussão sobre o Algoritmo k-NN

Vantagens Principais

- Simples de implementar e entender intuitivamente
- Não faz suposições sobre distribuição dos dados
- Adapta-se naturalmente a múltiplas classes
- Eficaz em espaços de baixa dimensionalidade
- Atualização trivial com novos dados

Otimizações

- Estruturas de indexação (KD-trees, Ball trees)
- Redução de dimensionalidade
- Seleção de features relevantes

Variações

- k-NN ponderado por distância
- k-NN com métricas customizadas
- Radius neighbors

Desafios e Limitações

- Computacionalmente custoso para grandes datasets
- Sensível à escolha da métrica de distância
- Sofre com a "maldição da dimensionalidade"
- Requer normalização/escalonamento de features
- Armazenamento de todos os dados de treinamento

Aplicações Típicas

- Sistemas de recomendação
- Reconhecimento de padrões
- Detecção de anomalias

O k-NN permanece como uma ferramenta valiosa no arsenal de ciência de dados, especialmente útil como baseline e em cenários onde a interpretabilidade e simplicidade são prioritárias. Sua natureza não-paramétrica o torna particularmente atraente quando os padrões nos dados são complexos e não seguem distribuições conhecidas.

Referências Principais

Esta seleção de referências representa os pilares fundamentais para o estudo aprofundado de mineração de dados, cobrindo desde conceitos básicos até técnicas avançadas e aplicações contemporâneas.



1. **J. Han, J. Pei, and H. Tong**, *Data Mining: Concepts and Techniques*, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.
2. **G. M. James, D. Witten, T. Hastie, and R. Tibshirani**, *An Introduction to Statistical Learning: With Applications in R*. Springer Nature, 2021.
3. **S. Garcia, J. Luengo, and F. Herrera**, *Data Preprocessing in Data Mining*. Springer, 2014.
4. **C. M. Bishop and H. Bishop**, *Deep Learning: Foundations and Concepts*. Springer Nature, 2023.